

```

import pandas as pd

from sqlalchemy.engine import URL

from sqlalchemy import create_engine

connection_url = URL.create(
    "postgresql",
    username="postgres",
    password="supermom@1",
    host="localhost",
    port=5432,
    database="postgres" # <-- Changed to your actual database name
)

# 1. Create the engine from your URL object
engine = create_engine(connection_url)

# 2. Pass the engine into pandas to run the query
df = pd.read_sql("SELECT * FROM customer", engine)

# View the first 5 rows to confirm it works
print(df.head())

import pandas as pd

import plotly.express as px

import streamlit as st

import warnings

warnings.filterwarnings("ignore")

```

```
st.set_page_config(page_title="Global E-commerce", page_icon=":bar_chart:",  
layout="wide")
```

```
st.title(":bar_chart: Global E-commerce")
```

```
# 1. Convert your date column to proper datetime format
```

```
df['order_date'] = pd.to_datetime(df['order_date'])
```

```
# 2. Create 3 horizontal columns for the slicers
```

```
filter_col1, filter_col2, filter_col3 = st.columns(3, gap="large")
```

```
# 3. Build the Slicers
```

```
with filter_col1:
```

```
    region_filter = st.multiselect(  
        "Select Region:",  
        options=df["region"].unique(),  
        default=df["region"].unique()  
    )
```

```
with filter_col2:
```

```
    # Set min and max dates for the calendar range
```

```
    min_date = df['order_date'].min().date()
```

```
    max_date = df['order_date'].max().date()
```

```
    date_range = st.date_input(  
        "Select Date Range:",  
        value=(min_date, max_date),  
        min_value=min_date,  
        max_value=max_date  
    )
```

with filter_col3:

```
# Assuming your column is named 'segment' (change if it is 'customer_segment')
customer_segment_filter = st.multiselect(
    "Select customer Segment:",
    options=df["customer_segment"].unique(),
    default=df["customer_segment"].unique()
)
```

4. Apply the selected filters to overwrite the dataframe

if len(date_range) == 2: # Ensures the script doesn't crash if only one date is clicked

```
start_date, end_date = date_range
df = df[
    (df["region"].isin(region_filter)) &
    (df["customer_segment"].isin(customer_segment_filter)) &
    (df["order_date"].dt.date >= start_date) &
    (df["order_date"].dt.date <= end_date)
]
```

4. Apply the selected filters to overwrite the dataframe

if len(date_range) == 2: # Ensures the script doesn't crash if only one date is clicked

```
start_date, end_date = date_range
df = df[
    (df["region"].isin(region_filter)) &
    (df["customer_segment"].isin(customer_segment_filter)) &
    (df["order_date"].dt.date >= start_date) &
    (df["order_date"].dt.date <= end_date)
]
```

```

# ---> ADD THIS EMPTY STATE CHECK HERE <---

if df.empty:

    st.warning("No data available. Please select at least one region, date range, or
segment.")

    st.stop() # This stops the rest of the script from running, preventing errors

st.markdown("----") # Adds a divider below the slicers before your scorecard


# Calculate margin percentage

df['margin'] = (df['profit'] / df['total_sales']) * 100


# Create 4 columns for the top scorecard

col1, col2, col3, col4 = st.columns(4, gap="medium")


col1.metric("Total Revenue", f"${df['total_sales'].sum():.2f}")
col2.metric("Total Profit", f"${df['profit'].sum():.2f}")
col3.metric("Average Margin", f"{df['margin'].mean():.1f}%")
col4.metric("Total Orders", f"{len(df)}")


st.markdown("----") # Adds a visual divider lines


# 1. Extract the Year and Month from the order_date

df['year_month'] = df['order_date'].dt.strftime('%Y-%m')


# 2. Group the data to get total sales and profit per month

trend_data = df.groupby('year_month')[['total_sales', 'profit']].sum().reset_index()

```

```
# 4. Build and display the line chart in the left column
```

```
# Create a 2-column layout for the bottom half of the dashboard
```

```
chart_col1, chart_col2 = st.columns(2)
```

```
# Build and display the line chart in the left column
```

```
with chart_col1:
```

```
    st.subheader("Revenue & Profit Trends")
```

```
    fig_line = px.line(
```

```
        trend_data,
```

```
        x='year_month',
```

```
        y=['total_sales', 'profit'],
```

```
        labels={'year_month': 'Month', 'value': 'Amount ($)', 'variable': 'Metric'},
```

```
        color_discrete_map={'total_sales': 'blue', 'profit': 'green'})
```

```
    )
```

```
    fig_line.update_layout(
```

```
        showlegend=True,
```

```
        legend=dict(
```

```
            orientation="h",
```

```
            yanchor="bottom",
```

```
            y=1.02,
```

```
            xanchor="right",
```

```
            x=1
```

```
        )
```

```
    )
```

```
    fig_line.update_traces(line=dict(width=3))
```

```
    st.plotly_chart(fig_line, use_container_width=True, config={"displayModeBar": False})
```

```
# Build and display the bar chart in the right column
```

with chart_col2:

```
st.subheader("Profit by Product Category")

category_profit = df.groupby('product_category')['profit'].sum().reset_index()

fig_bar = px.bar(
    category_profit,
    x='profit',
    y='product_category',
    orientation='h',
    labels={'profit': 'Total Profit ($)', 'product_category': 'Category'},
    color_discrete_sequence=['#1f77b4'], # Forces a single uniform blue color
    text_auto='.2s'          # Adds exact profit values to the end of each bar
)

st.plotly_chart(fig_bar, use_container_width=True, config={"displayModeBar": False})
```

1. Group data by country and calculate metrics

```
country_data = df.groupby('country').agg(
    sales_volume=('total_sales', 'count'), # Counts number of orders
    total_revenue=('total_sales', 'sum'),
    total_profit=('profit', 'sum')
).reset_index()
```

Calculate actual net margin percentage per country

```
country_data['net_margin'] = (country_data['total_profit'] /
country_data['total_revenue']) * 100
```

2. Create a new 2-column row underneath your current charts

```
map_col1, map_col2 = st.columns(2)
```

3. Build and display the map in the left column

with map_col1:

```
st.subheader("Global Sales Volume")
```

```
fig_map = px.choropleth(  
    country_data,  
    locations='country',  
    locationmode='country names',  
    color='sales_volume',  
    hover_name='country',  
    hover_data={  
        'country': False, # Prevents the country name from showing twice in the tooltip  
        'sales_volume': True,  
        'total_revenue': ':.2f',  
        'net_margin': ':.1f%'  
    },  
    color_continuous_scale='Blues',  
    labels={  
        'sales_volume': 'Volume',  
        'total_revenue': 'Revenue ($)',  
        'net_margin': 'Net Margin (%)'  
    }  
)
```

Adjust map styling to fit cleanly in the dashboard

```
fig_map.update_geos(showframe=False, showcoastlines=True,  
projection_type='equiarectangular')
```

```
st.plotly_chart(fig_map, use_container_width=True, config={"displayModeBar": False})
```

4. Build and display the scatter plot in the right column

with map_col2:

```
st.subheader("Profit vs. Discount Impact")
```

```
fig_scatter = px.scatter(  
    df,  
    x='discount_percent', # Change to 'discount' if your SQL column is named differently  
    y='profit',  
    hover_name='order_id', # Makes the Order ID the bold title of the tooltip  
    color='profit',  
    color_continuous_scale='RdYlGn', # Red for negative profit, Green for positive  
    labels={  
        'discount_percent': 'Discount (%)',  
        'profit': 'Profit ($)'  
    }  
)
```

Adds a dotted red line at \$0 to easily spot orders that lost money

```
fig_scatter.add_hline(y=0, line_dash="dot", line_color="red", line_width=1)  
fig_scatter.update_layout(  
    height=500, # Adjusts the height of the scatter plot for better visibility  
    margin=dict(l=40, r=40, t=40, b=80), # Reduces whitespace around the chart  
    xaxis=dict(  
        title='Discount (%)',  
        title_standoff=25  
    ),
```

```

yaxis=dict(
    title="Profit ($)" # Sets ticks every 5% for clarity
)
)

st.plotly_chart(fig_scatter, use_container_width=True,config={"displayModeBar":
False})

st.markdown("---") # Adds a visual divider above the pie chart
st.subheader("Revenue by Payment Method")

# 1. Group the data to calculate total revenue per payment method
payment_data = df.groupby('payment_method')['total_sales'].sum().reset_index()

# 2. Create the pie chart
fig_pie = px.pie(
    payment_data,
    values='total_sales',
    names='payment_method',
    hole=0.4, # Optional: Turns the pie chart into a modern donut chart
    labels={'total_sales': 'Total Revenue ($)', 'payment_method': 'Payment Method'}
)

# 3. Format the labels to show cleanly inside the slices
fig_pie.update_traces(textposition='inside', textinfo='percent+label')

st.plotly_chart(fig_pie, use_container_width=True,config={"displayModeBar": False})

```

